

REPORT DEL PROGETTO D'ESAME DEL CORSO DI "COMPUTER VISION AND PATTERN RECOGNITION"

PRESENTAZIONE DEL PROGETTO: DESCRIZIONE DEL PROBLEMA E OBIETTIVI

L'**obiettivo** centrale di questo elaborato è affrontare un classico problema di *Scene Recognition* (Riconoscimento di Scene) nel campo della Computer Vision, costruendo, analizzando e ottimizzando classificatori di immagini basati su **Reti Neurali Convoluzionali (CNN)**.

Il dominio applicativo è definito da un dataset di riferimento (*Lazebnik et al., 2006*) che cataloga le immagini in 15 categorie: queste spaziano da ambienti interni (come uffici, cucine, salotti, camere da letto e magazzini), a scenari urbani e infrastrutturali (edifici industriali, grattacieli, centri storici, strade, autostrade e periferie), fino a contesti prettamente naturali (coste, paesaggi aperti, montagne e foreste). I dati forniti vengono inoltre suddivisi in un set dedicato all'addestramento e uno destinato al test finale.

Il lavoro è strutturato in tre fasi, che permettono di esplorare e risolvere progressivamente le criticità tipiche dell'addestramento di reti neurali, quali la **scarsità dei dati** e l'**overfitting**:

1. **Definizione di un modello di base (Baseline):** La prima fase consiste nel creare da zero una CNN superficiale ("*shallow*") seguendo vincoli architetturali estremamente rigidi. È richiesto di processare immagini ridimensionate a 64×64 pixel, partizionare il training set fornito ricavando un 15% di dati di validazione e applicare una specifica inizializzazione gaussiana dei pesi per addestrare il modello tramite **discesa del gradiente stocastico (SGD)**. Lo scopo di questa fase non è massimizzare le prestazioni, ma validare l'apprendimento iniziale raggiungendo un'accuratezza attesa di circa il 30%.
2. **Ottimizzazione e Regolarizzazione:** La seconda fase del problema richiede di contrastare i limiti prestazionali intrinseci della rete di base. Il compito prevede l'integrazione selettiva di strategie avanzate, quali l'arricchimento del dataset di addestramento tramite **data augmentation** (come le riflessioni speculari sinistra-destra), l'introduzione di layer **di batch normalization** o **dropout**, e l'affinamento dell'ottimizzazione (ad esempio passando all'**algoritmo Adam**). L'obiettivo di queste modifiche è raddoppiare la capacità predittiva del modello, puntando a un'accuratezza del 60%.
3. **Applicazione del Transfer Learning:** L'ultima sfida affronta il problema della classificazione sfruttando la conoscenza pregressa di reti profonde (come AlexNet). Il progetto impone di raggiungere un'accuratezza superiore all'85% esplorando due paradigmi distinti: il **fine-tuning** (congelando i pesi pre-addestrati e riaddestrando esclusivamente l'ultimo strato lineare sulle 15 classi del problema) e la **Feature Extraction** (utilizzando le attivazioni intermedie della rete per alimentare un classificatore esterno, specificamente una Support Vector Machine lineare multiclasse).

PARTE 1: DEFINIZIONE DI UN MODELLO BASE (BASELINE)

Introduzione e Pre-Processing delle immagini

Il primo step del progetto consiste nello sviluppo e nell'addestramento da zero di una rete convoluzionale *shallow* (superficiale), in rigoroso accordo con le specifiche architetture e i vincoli imposti dalla consegna.

La **fase di pre-processing** è cruciale per uniformare i dati in ingresso alla rete. Tramite i moduli della libreria `torchvision.transforms`, le immagini sono state convertite in scala di grigi (a singolo canale) e sottoposte a un rescaling forzato alla risoluzione di 64×64 pixel.

In conformità con le specifiche, si è scelto deliberatamente di evitare la normalizzazione standard (che mapperebbe i pixel nell'intervallo $[0,1]$). I valori sono stati mantenuti nel loro intervallo nativo $[0,255]$ utilizzando un'apposita funzione `Lambda`, in modo da non vanificare gli effetti della successiva inizializzazione gaussiana dei pesi.

```
# definizione del preprocessing
transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1), # (scala di grigi)
    transforms.Resize((64, 64)),               # Rescaling forzato a 64x64
    transforms.ToTensor(),                      # Converte l'immagine in un tensore (valori [0.0, 1.0])
    transforms.Lambda(lambda x: x * 255.0)      # Moltiplica per 255 per riportare i pixel nel range [0, 255]
])
```

Suddivisione in Training Set e Validation Set

Per consentire un monitoraggio oggettivo delle metriche durante l'addestramento, il training set originale è stato partizionato dinamicamente. Sfruttando la funzione `random_split`, l'85% dei campioni è stato destinato all'effettivo aggiornamento dei pesi (**Training Set**), mentre il restante 15% è stato isolato per misurare la reale capacità di generalizzazione del modello (**Validation Set**).

I dati sono stati successivamente impacchettati in **minibatch** da 32 immagini tramite il modulo `DataLoader`. Il mescolamento casuale (`shuffle=True`) è stato attivato solo per il set di addestramento, in modo da rompere la sequenzialità dei dati e stabilizzare la discesa del gradiente.

Architettura della Rete Neurale Convoluzionale

L'architettura della rete è stata definita basandosi sul layout tabellare fornito nella consegna. La classe `BaseLineCNN` è composta da un'alternanza di tre blocchi di estrazione delle feature:

- livelli convoluzionali 3×3
- strati di attivazione **ReLU**
- moduli di **Max Pooling** 2×2

seguiti da un singolo strato densamente connesso (*Fully Connected*) che funge da classificatore finale.

OSS. → È stata appositamente omessa la funzione di attivazione **Softmax** in uscita, poiché essa è già integrata internamente dalla funzione di costo `CrossEntropyLoss` di PyTorch.

```

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        # Convoluzione 1: 8 filtri 3x3. Uso padding=1 per mantenere la dim spaziale 64x64 prima del pooling
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=8, kernel_size=3, stride=1, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2) # Dimezza a 32x32

        # Convoluzione 2: 16 filtri 3x3
        self.conv2 = nn.Conv2d(in_channels=8, out_channels=16, kernel_size=3, stride=1, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2) # Dimezza a 16x16

        # Convoluzione 3: 32 filtri 3x3
        self.conv3 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1)
        self.relu3 = nn.ReLU()
        self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2) # Dimezza a 8x8

        # Layer Fully Connected.
        # Dimensioni in ingresso: 32 canali * 8 altezza * 8 larghezza = 2048 feature
        self.fc = nn.Linear(in_features=32 * 8 * 8, out_features=15)

        # Inizializzazione pesi
        self._initialize_weights()

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = self.pool3(self.relu3(self.conv3(x)))
        x = torch.flatten(x, 1) # Appiattisce i tensori spaziali in un vettore
        x = self.fc(x)
        return x # NESSUN SOFTMAX QUI

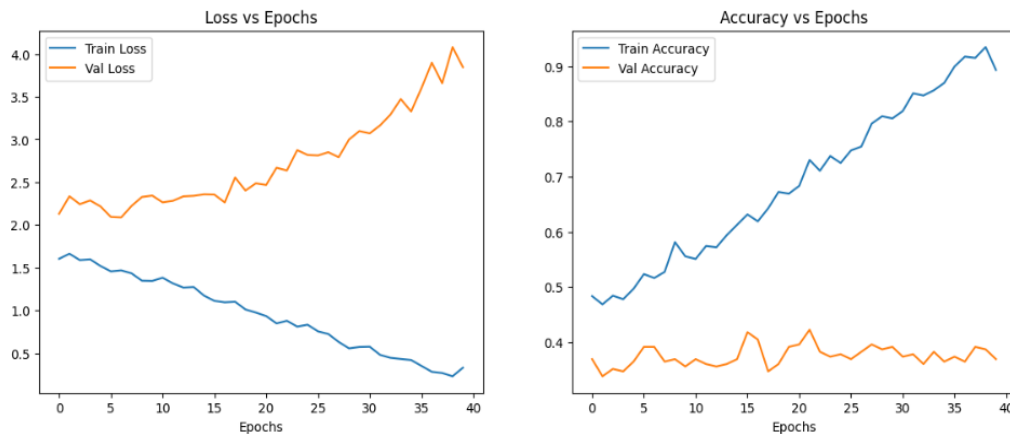
```

Inizializzazione e Ottimizzazione

Come previsto dai requisiti, i pesi dei livelli convoluzionali e lineari sono stati campionati da una distribuzione gaussiana a media nulla e deviazione standard $\sigma = 0.01$, con i bias impostati a zero. Il motore di calcolo impiegato è l'ottimizzatore **Stochastic Gradient Descent (SGD)** configurato con **momentum** a 0.9. A causa del range elevato dei pixel di input ($[0,255]$), il learning rate è stato empiricamente tarato sul valore di 10^{-4} per garantire stabilità ed evitare l'esplosione dei gradienti nei primi cicli.

Analisi dei Risultati e Valutazione Finale: Curve di Apprendimento

I grafici relativi all'andamento della **Loss** e dell'**Accuracy** (riportati di seguito) illustrano in modo inequivocabile le dinamiche di addestramento della rete, confermando le limitazioni teoriche di un'architettura sprovvista di regolarizzazione.



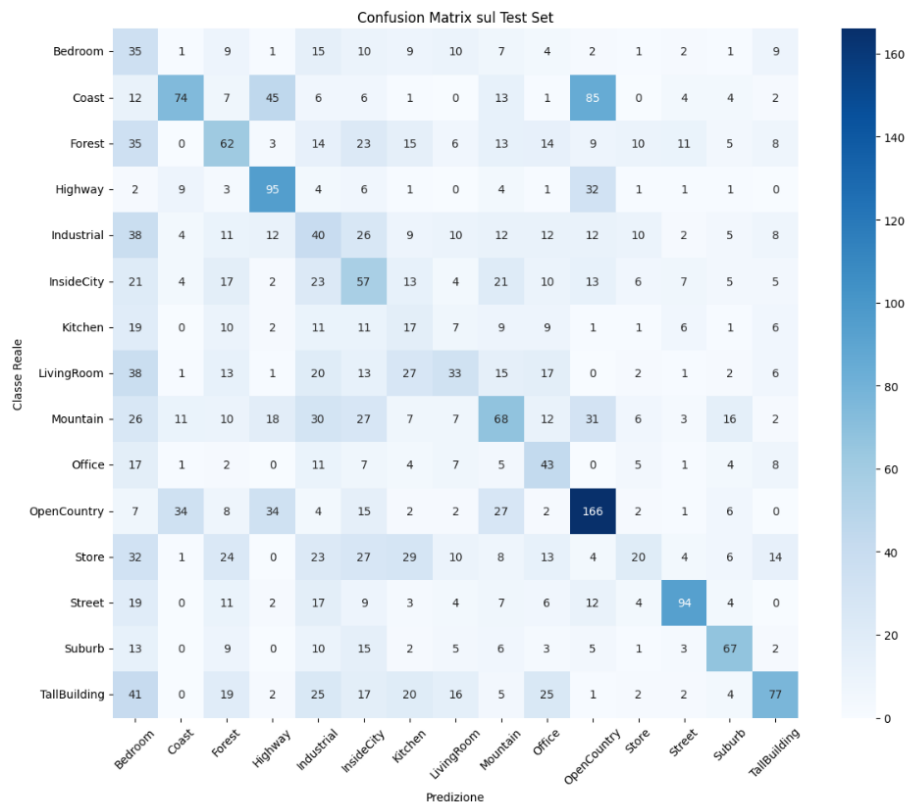
Si osserva una marcata e progressiva divergenza tra le metriche calcolate sul training set e quelle sul validation set. Mentre la **Training Loss** decresce in modo quasi lineare per tutte le 40 epoche fino a valori prossimi allo 0.2, la **Validation Loss** inverte la propria tendenza già intorno all'epoca 6, impennandosi progressivamente fino a superare la soglia del 4.0.

L'andamento dell'accuratezza è speculare: la rete arriva a memorizzare quasi perfettamente il set di addestramento (superando il 90% di **Training Accuracy**), mentre la **Validation Accuracy** si satura istantaneamente, oscillando in una banda stazionaria ristretta tra il 35% e il 40%.

Questo delta prestazionale rappresenta un caso emblematico di **overfitting** (sovradattamento): i filtri convoluzionali hanno esaurito la propria capacità di generalizzazione semantica, iniziando a mappare il rumore statistico e le specifiche distribuzioni di pixel del training set piuttosto che estrarre feature visive universali.

Analisi dei Risultati e Valutazione Finale: Matrice di Confusione

Nonostante la marcata tendenza all'overfitting, il ripristino dei pesi ottimali (tramite **early stopping**) ha permesso al classificatore di raggiungere un'**accuratezza complessiva del 31.76%** sul test set indipendente. Tale risultato soddisfa pienamente il requisito di baseline imposto per questa prima fase operativa (fissato attorno al 30%).



Un'analisi dettagliata della matrice di confusione evidenzia i bias predittivi e i limiti estrattivi di questa configurazione spaziale:

- Il modello mostra una maggiore solidità predittiva su categorie caratterizzate da pattern macroscopici o geometrie globali ricorrenti. Sulla diagonale principale spiccano i 166 veri positivi per la classe **OpenCountry**, seguiti dalle buone prestazioni sulle categorie infrastrutturali come **Highway** (95 veri positivi), **Street** (94) e **TallBuilding** (77).
- Emergono severe criticità nella discriminazione di scene che presentano texture simili. L'errore sistematico più rilevante si verifica tra le categorie naturali: ben 85 campioni appartenenti alla classe reale **Coast** vengono erroneamente classificati come **OpenCountry**. Questo indica un'incapacità della rete di distinguere orizzonti piatti (tipici di entrambe le classi) senza l'ausilio dell'informazione cromatica.
- In situazioni di incertezza, il modello tende a distorcere le predizioni verso classi affini. Osservando la prima colonna della matrice, si nota come la classe **Bedroom** raccolga un numero anomalo di falsi positivi provenienti da ambienti molto eterogenei, tra cui **LivingRoom** (38), **Industrial** (38) e persino categorie outdoor come **Forest** (35).

OSS.

In sintesi, i falsi positivi fortemente distribuiti al di fuori della diagonale principale confermano la necessità di introdurre strati di regolarizzazione (Dropout, Batch Normalization) e tecniche di arricchimento dei dati (Data Augmentation) per irrobustire la capacità discriminativa dei layer convoluzionali.

PARTE 2: OTTIMIZZAZIONE E REGOLARIZZAZIONE DELLA RETE

Introduzione e Implementazione

Nella seconda fase, l'obiettivo del progetto prevede l'implementazione di tecniche avanzate per arginare la divergenza (overfitting) riscontrata nel modello di base e raddoppiare le capacità predittive della rete, puntando a un'accuratezza di circa il 60%. Per raggiungere tale scopo, l'architettura e la pipeline di pre-processing sono state irrobustite sfruttando specifiche funzioni del framework PyTorch.

In primo luogo, si è intervenuti sul training set introducendo la **Data Augmentation** per incrementare artificialmente la variabilità dei campioni. Sfruttando la stringa di codice `transforms.RandomHorizontalFlip(p=0.5)`, le immagini in fase di addestramento sono state sottoposte a una riflessione speculare (sinistra-destra) con probabilità del 50%.

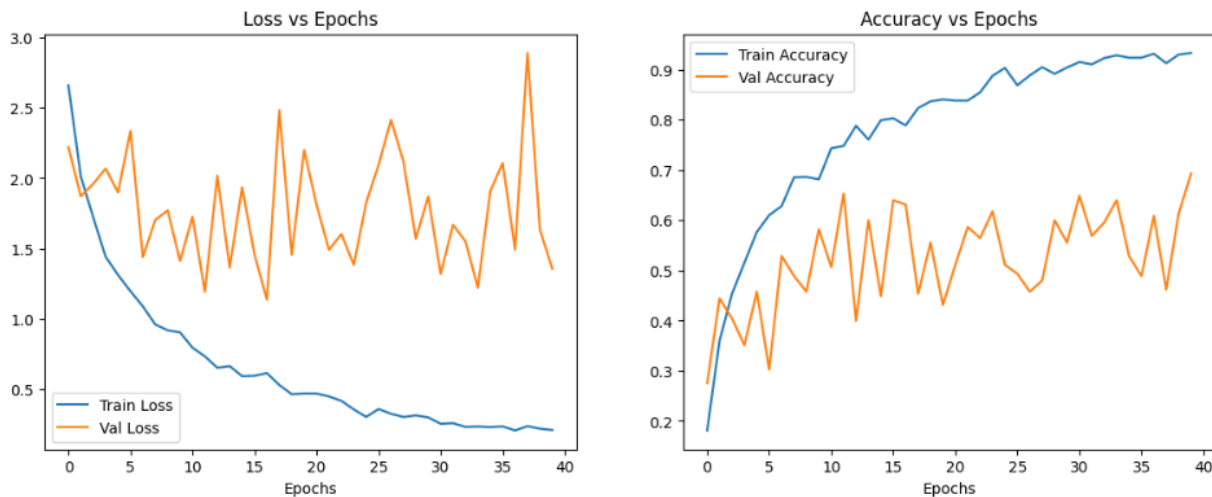
OSS.

Come da consegna, questa trasformazione è stata omessa nei dataloader di validazione e test.

L'architettura originaria è stata potenziata sia in capacità espressiva (raddoppiando il numero dei filtri convoluzionali nei tre blocchi: $16 \rightarrow 32 \rightarrow 64$) sia in stabilità. Sono stati introdotti strati di normalizzazione tramite la funzione `nn.BatchNorm2d()` posizionati immediatamente prima di ogni attivazione `nn.ReLU()`, al fine di ridurre il **covariate shift interno**. Inoltre, per ridurre l'errore di test, è stato inserito un livello di regolarizzazione `nn.Dropout(p=0.5)` prima del classificatore lineare finale. Infine, l'**ottimizzatore SGD** è stato sostituito passando all'algoritmo adattivo `optim.Adam()` per garantire una discesa del gradiente più efficiente.

Analisi dei Risultati e Valutazione Finale: Curve di Apprendimento

L'osservazione dei grafici generati per la Fase 2 (presentati di seguito) restituisce un quadro dinamico radicalmente diverso rispetto alla *baseline*.



La **Training Loss** decresce in modo stabile, scendendo dal valore iniziale di circa 2.6 fino a stabilizzarsi in prossimità dello 0.2, con una **Training Accuracy** che supera agevolmente la soglia del 90%.

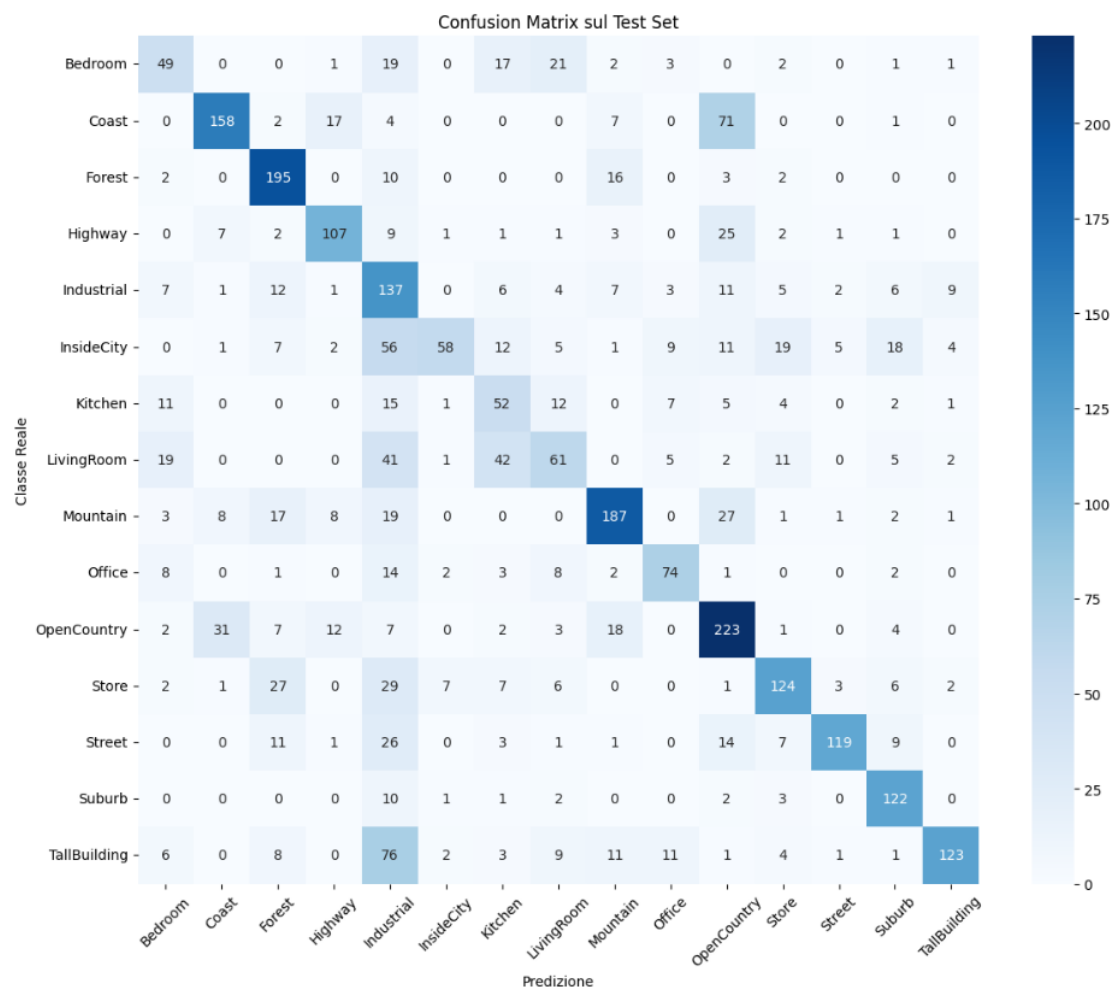
Il comportamento per quanto riguarda la validazione, tuttavia, si presenta estremamente rumoroso: la **Validation Loss** è caratterizzata da oscillazioni ad alta frequenza (con picchi che sfiorano il 2.9 e minimi attorno all'1.1), mentre la **Validation Accuracy** fluttua tra il 30% e il picco del 65%.

Questa instabilità geometrica della curva di validazione è un fenomeno documentato quando si combinano ottimizzatori adattivi (come **Adam**) con tecniche che introducono forte stocasticità (il **Dropout** e la **RandomHorizontalFlip**), specialmente su dataset di dimensioni modeste elaborati con minibatch ridotti. Nonostante l'ampia varianza, la divergenza monotona verso l'alto (tipica dell'overfitting visto nella Parte 1) risulta mitigata. Il meccanismo di **early stopping** implementato tramite la funzione `model.load_state_dict(best_model_weights)` ha permesso di isolare e salvare i parametri corrispondenti al minimo della loss di validazione (attorno all'epoca 16), scartando il rumore delle epoche successive.

Analisi dei Risultati e Valutazione Finale: Matrice di Confusione

Il collaudo finale sul test set indipendente ha restituito un'**accuratezza complessiva del 59.93%**, un risultato che centra il target del ~60% richiesto dalle specifiche di progetto.

L'analisi della matrice di confusione rivela come la combinazione di **Batch Normalization** e **Data Augmentation** abbia stravolto positivamente la capacità estrattiva della rete, innalzando drasticamente i veri positivi lungo la diagonale principale, specialmente per le categorie *outdoor*. Spiccano i risultati su **OpenCountry** (223 predizioni corrette), **Forest** (195), **Mountain** (187) e **Coast** (158). Anche i contesti urbani mostrano miglioramenti sostanziali, con **Industrial**, **Store**, **Street**, **Suburb** e **TallBuilding** tutte saldamente sopra i 110 campioni correttamente classificati.



OSS.

Nonostante il raddoppio dell'accuratezza complessiva, il modello evidenzia delle vulnerabilità dovute a dei “**colli di bottiglia**”:

1. La rete fatica a separare le **categorie di interni**. La classe **Bedroom** (49 veri positivi) disperde i suoi campioni verso **LivingRoom** (21 errori) e **Industrial** (19 errori). Discorso analogo per **Kitchen** (52 veri positivi) e **LivingRoom** (61 veri positivi), le cui predizioni collassano di frequente sulla classe **Industrial** (rispettivamente 15 e 41 falsi negativi). Questo limite è intrinsecamente legato all'assenza di informazioni cromatiche (la rete elabora tensori in scala di grigi ad un solo canale) e alla bassa risoluzione (64×64), che rendono elementi strutturali come mobili, tavoli o macchinari indistinguibili.
2. Si riscontra un errore di classificazione tra **InsideCity** (solo 58 veri positivi) e **Industrial** (ben 56 falsi negativi provenienti da InsideCity), indicando l'incapacità di discernere l'edilizia residenziale da quella commerciale/industriale. Permane, inoltre, il bias paesaggistico già visto in Fase 1: 71 immagini appartenenti alla classe **Coast** vengono classificate come **OpenCountry**.

In conclusione, le metriche certificano il pieno successo della regolarizzazione architettuale: la rete estrae ora feature spaziali robuste, sebbene la compressione e l'assenza del colore precludano una segmentazione perfetta delle classi dotate di architetture o texture visivamente sovrapponibili.

PARTE 3: APPLICAZIONE DEL TRANSFER LEARNING TRAMITE RETE PRE-ADDESTRATA (AlexNet)

Lineare Introduzione e Riallineamento del Pre-Processing

In questa ultima fase del progetto abbandoniamo l'addestramento "da zero" per utilizzare il paradigma del **Transfer Learning**, con l'obiettivo di superare la soglia dell'**85% di accuratezza sul test set**. Sfruttando la rete convoluzionale profonda **AlexNet**, pre-addestrata sull'enorme dataset ImageNet, il modello eredita una gerarchia di feature visive universali (bordi, gradienti, texture complesse) già consolidate.

L'adozione di un'architettura standardizzata ha richiesto una profonda revisione della pipeline di pre-processing. Le immagini non sono state più convertite in scala di grigi, ma elaborate nel formato RGB (a 3 canali) e ridimensionate alla risoluzione geometrica di 224×224 pixel richiesta da AlexNet. Elemento cruciale di questa fase è stata l'introduzione della normalizzazione tramite il modulo `transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`, indispensabile per allineare la distribuzione dei tensori di input a quella originaria di ImageNet sui quali, i filtri della rete, sono stati originariamente ottimizzati.

Abbiamo, come richiesto dalla consegna, adottato due metodologie distinte per sfruttare il potenziale di questa architettura.

Approccio 1: Fine-Tuning del Classificatore

Nel primo approccio, la topologia di AlexNet è stata importata (come si evince dal download dei pesi `alexnet-owt-7be5be79.pth` nei log di esecuzione) ma "congelata" nella sua sezione convoluzionale profonda. Attraverso il ciclo `for param in model_ft.parameters(): param.requires_grad = False`, il calcolo del gradiente è stato inibito per tutti i livelli di estrazione delle feature.

Le metriche di addestramento (eseguito per sole 10 epoche per prevenire l'overfitting locale del singolo strato) certificano una veloce convergenza:

- Già all'Epoca 1, la **Training Accuracy** si attesta al 69.25%, un valore superiore a qualsiasi prestazione raggiunta al termine delle 40 epoche nella Fase 2.
- All'Epoca 4, l'accuratezza in addestramento supera il 95%, per saturarsi asintoticamente sul 98.51% all'Epoca 10.

```
Epoch 1/10 | Train Acc: 69.25%
Epoch 2/10 | Train Acc: 90.51%
Epoch 3/10 | Train Acc: 93.02%
Epoch 4/10 | Train Acc: 95.84%
Epoch 5/10 | Train Acc: 95.84%
Epoch 6/10 | Train Acc: 97.18%
Epoch 7/10 | Train Acc: 97.73%
Epoch 8/10 | Train Acc: 98.20%
Epoch 9/10 | Train Acc: 98.27%
Epoch 10/10 | Train Acc: 98.51%
Test Accuracy Fine-Tuning: 86.06% (Obiettivo > 85%)
```

La valutazione finale ha restituito una **Test Accuracy dell'86.06%**, centrando e superando l'obiettivo di progetto (fissato a $> 85\%$). Questo risultato dimostra empiricamente che i filtri ereditati da ImageNet costituiscono una base talmente robusta da rendere l'addestramento del solo classificatore finale sufficiente per mappare correttamente le 15 categorie del problema.

Approccio 2: Feature Extraction e Support Vector Machine (SVM)

Nel secondo approccio, AlexNet è stata utilizzata esclusivamente come un estrattore di caratteristiche (**Feature Extractor**), rinunciando del tutto al calcolo dei gradienti e all'uso di reti neurali per la fase di predizione finale.

Sostituendo l'ultimo livello della rete con una funzione identità (`nn.Identity()`), ogni immagine del dataset è stata proiettata nel penultimo livello di AlexNet. L'output di questa operazione, come riportato dai log, ha generato due matrici ad alta dimensionalità:

- **Matrice di Train:** dimensioni (1275,4096)
- **Matrice di Test:** dimensioni (2985,4096)

```
Dimensioni feature estratte: Train (1275, 4096), Test (2985, 4096)
Test Accuracy SVM Multiclasse: 86.23% (Obiettivo > 85%)
```

Ogni singola immagine è stata quindi "compressa" in un vettore rappresentativo di 4096 feature numeriche. Su questi dati estratti è stato addestrato un modello di Machine Learning classico, ovvero una Support Vector Machine (SVM) lineare multiclasse (**LinearSVC** di **scikit-learn**).

La validazione sul set di test ha prodotto una **Test Accuracy dell'86.23%**. Questo risultato non solo supera l'obiettivo minimo dell'85%, ma si rivela marginalmente superiore all'approccio di fine-tuning (86.23% contro 86.06%)

CONSIDERAZIONE FINALE

L'implementazione del **Transfer Learning** fornisce un salto prestazionale notevole del progetto. Passando dal $\sim 31\%$ della baseline iniziale, al $\sim 60\%$ della rete regolarizzata, fino a sfondare il muro dell'86% con AlexNet, si evince, pertanto, il limite delle reti *shallow* addestrate da zero su insiemi di dati esigui. Il trasferimento di conoscenza si presenta come un'ottima strategia per compiti di Computer Vision: delegando l'estrazione delle feature a reti pre-addestrate su enormi dataset, è possibile risolvere problemi di classificazione complessi con eccezionale precisione e costi computazionali ridotti al minimo.